

Final Report
Binary Analysis with AI

Montassar Arfaoui

March 2026

0.1 Introduction

This report presents the final outcome of the **Binary Analysis with AI** practical course project. The focus of this project was the reproduction, modification, and evaluation of **PalmTree**, a pre-trained assembly language model designed for instruction embedding.

This report focuses on the conceptual contributions, implementation decisions, experimental setup, and interpretation of results. Detailed scripts, commands, and execution instructions are available in the accompanying repository and README file.

The original PalmTree is a pre-trained assembly language model designed to learn instruction embeddings for binary analysis tasks. It is built on top of a lightweight BERT-style transformer architecture and trained in a self-supervised manner on large collections of compiled binaries.

PalmTree improves over earlier approaches such as word2vec and Asm2Vec by explicitly modeling both syntactic and semantic relationships. It achieves this using three training objectives:

1. **Masked Language Modeling (MLM)**: Predict masked tokens within an instruction to learn syntax.
2. **Context Window Prediction (CWP)**: Predict whether two instructions appear close in a basic block, capturing control-flow structure.
3. **Def-Use Prediction (DUP)**: Predict whether one instruction defines a value that another instruction later uses, modeling data-flow dependencies.

Although PalmTree is open source, its preprocessing pipeline depends on Binary Ninja, a commercial reverse-engineering tool. This dependency restricts reproducibility and limits accessibility in academic environments.

0.2 Project Objectives

The project was structured around two primary objectives:

0.2.1 Goal 1: Fully Open-Source PalmTree Pipeline

The first goal was to rebuild PalmTree’s preprocessing pipeline using only open-source tools. The required components include:

- Binary disassembly
- Basic block construction
- Instruction tokenization

- Control-flow pair generation (CWP)
- Def-use pair generation (DUP)

Binary Ninja was replaced with **angr**, an open-source binary analysis framework. This ensures that the entire pipeline from raw binaries to training data is reproducible without proprietary software.

0.2.2 Goal 2: Ablation Study of Training Objectives

The second objective was to analyze how each training objective contributes to overall performance. The following variants were trained:

Model Variant	MLM	CWP	DUP
PalmTree-M	X	-	-
PalmTree-MD	X	-	X
PalmTree-MC	X	X	-
Full PalmTree	X	X	X

This design isolates the effect of CWP and DUP when combined with MLM.

0.3 Implementation Details

0.3.1 Binary Disassembly and Pair Generation

The first step was implementing binary disassembly using angr. From raw x86-64 binaries:

- Control Flow Graphs (CFGs) were extracted.
- Data Flow Graphs (DFGs) were constructed.
- Instruction pairs were derived from CFG edges (for CWP).
- Def-use pairs were derived from DFG edges (for DUP).

Def-use relations connect instructions that define a value (e.g., writing to a register) to later instructions that use that value, capturing semantic dependencies beyond mere positional proximity.

The tokenization strategy followed PalmTree’s fine-grained approach, preserving symbolic structure rather than treating instructions as plain text.

```

call rax      add rsp 8
add rsp 8     ret
add rsp 8     ret
sub rsp 8     mov rax qword ptr [ rip + address ]
mov rax qword ptr [ rip + address ]      test rax rax
test rax rax  je address
je address    call rax
call rax      add rsp 8
add rsp 8     ret
mov rax qword ptr [ rip + address ]      test rax rax
test rax rax  je address
je address    call rax
call rax      add rsp 8
add rsp 8     ret
test rax rax  je address
je address    add rsp 8
add rsp 8     ret
je address    call rax
call rax      add rsp 8
add rsp 8     ret
push qword ptr [ rip + address ] jmp qword ptr [ rip + address ]
xor ebp ebp  mov r9 rdx
mov r9 rdx   pop rsi
pop rsi mov rdx rsp
mov rdx rsp  and rsp address
and rsp address push rax
push rax     push rsp
push rsp     mov r8 symbol
mov r8 symbol mov rcx symbol
mov rcx symbol mov rdi symbol
mov rdi symbol call qword ptr [ rip + address ]
mov r9 rdx   pop rsi
pop rsi mov rdx rsp
mov rdx rsp  and rsp address
and rsp address push rax
push rax     push rsp
push rsp     mov r8 symbol
mov r8 symbol mov rcx symbol
mov rcx symbol mov rdi symbol
mov rdi symbol call qword ptr [ rip + address ]
pop rsi mov rdx rsp
mov rdx rsp  and rsp address
and rsp address push rax
push rax     push rsp
push rsp     mov r8 symbol
mov r8 symbol mov rcx symbol
mov rcx symbol mov rdi symbol
mov rdi symbol call qword ptr [ rip + address ]
mov rdx rsp  and rsp address
and rsp address push rax
push rax     push rsp
push rsp     mov r8 symbol
mov r8 symbol mov rcx symbol
mov rcx symbol mov rdi symbol
mov rdi symbol call qword ptr [ rip + address ]
and rsp address push rax
push rax     push rsp
push rsp     mov r8 symbol
mov r8 symbol mov rcx symbol

```

Figure 1: Example part of generated dfg pairs from the txt file

0.3.2 Environment and Data Separation

To ensure reproducibility and modularity, the project was divided into two conda environments:

- **angr environment:** input generation and graph extraction.
- **palmtree-train environment:** model training and evaluation (PyTorch).

The dataset was strictly separated:

- 5 projects for training
- 2 projects for evaluation

This prevents data leakage and ensures meaningful retrieval evaluation.

0.3.3 Tensor Preprocessing Optimization

Initial experiments revealed preprocessing bottlenecks during training. Tensors were being constructed dynamically inside the training loop, causing:

- Increased CPU usage
- Long training times
- Occasional out-of-memory errors

To resolve this:

- Preprocessing was moved offline.
- Fixed-length tensors were generated once.
- Tensors were persisted to disk and reused.

This significantly improved efficiency and training stability.

0.4 Evaluation Setup

Evaluation followed a retrieval-based design:

- Anchors: functions from a given project.
- Positives: the same function compiled under different compiler settings.
- Negatives: different functions.

Approximately 28,000 anchor functions were evaluated.
Metrics:

- Recall@10 measures whether at least one correct function appears in the top 10 most similar candidates.

[illegible]

Figure 2: The JSON file generated using angr represents basic blocks, each consisting of a straight-line sequence of instructions.



Figure 3: The embeddings obtained after running the how2use script.

0.5 Results and Analysis

0.5.1 Open-Source Pipeline Performance

The first research question asked whether PalmTree can remain effective without Binary Ninja.

The results show:

- Strong and stable retrieval performance.
- Comparable embedding quality.
- Reduced computational overhead due to offline preprocessing.

This confirms that a fully open-source pipeline using angr is both feasible and efficient.

```
[{
  "model": "embeddings_FullTraining_All.npy",
  "num_anchors_evaluated": 27845,
  "Recall@1": 0.8637816484108457,
  "Recall@10": 0.9444065361824385,
  "MRR": 0.8958475017384363,
  "NDCG": 0.7794988984522181,
  "NDCG@10": 0.7222652008951365
}]
(palmtree-train) ru94wid@arunika:~/projects/PalmTree$
```

Figure 4: Evaluation results

0.5.2 Ablation Study Findings

All model variants achieved strong retrieval performance. Observations include:

- Results are highly consistent across configurations.
- MLM + DUP slightly outperforms other reduced variants.
- The full model remains strongest overall.

Interpretation:

- MLM captures internal instruction structure.
- DUP contributes strong semantic data-flow signals.
- CWP adds contextual robustness to reordering.

The relatively small performance gap suggests architectural robustness. Even simplified variants maintain high-quality embeddings.

The ablation results reveal a clear performance trend across training variants. The configuration combining **MLM + CWP** consistently shows the weakest performance among all tested models. This suggests that modeling local control-flow proximity alone does not provide sufficiently strong semantic supervision when data-flow information is absent. In contrast, the **MLM + DUP** variant outperforms the other reduced configurations and achieves performance very close to the full PalmTree model. This indicates that def-use relationships contribute more directly to capturing semantic dependencies between instructions, leading to more discriminative embeddings. Overall, the results suggest that data-flow supervision (DUP) has a stronger impact on retrieval quality than control-flow supervision (CWP) when combined with MLM.

```

(palmtree-train) ru@pali:~/projects/palmtree/evaluation$ ls
anchors_pools.json      embeddings_FullTraining_MLH.fids.txt  embeddings_FullTraining_MLH_OUP.fids.txt  log_embed_FullTraining_All.log      log_eval_FullTraining_MLH.log
embed_functions.py      embeddings_FullTraining_MLH.npy       embeddings_FullTraining_MLH_OUP.npy       log_embed_FullTraining_MLH.log      log_embed_FullTraining_MLH_OUP.log
embeddings_FullTraining_All.fids.txt  embeddings_FullTraining_MLH_OUP.fids.txt  eval_retrieval.py                       log_embed_FullTraining_MLH_OUP.log
(palmtree-train) ru@pali:~/projects/palmtree/evaluation$ cd -
/home/ru@pali:~/projects/palmtree
(palmtree-train) ru@pali:~/projects/palmtree$ python evaluation/eval_retrieval.py \
--emb evaluation/embeddings_FullTraining_All.npy \
--pools evaluation/anchors_pools.json \
--out evaluation/results_FullTraining_All.json \
2>&& | tee evaluation/log_eval_FullTraining_All.log
{
  "model": "embeddings_FullTraining_All.npy",
  "num_anchors_evaluated": 27845,
  "Recall@81": 0.8637836484188457,
  "Recall@81@": 0.9444865361824385,
  "MRP": 0.0588475817394363
}
(palmtree-train) ru@pali:~/projects/palmtree$ python evaluation/eval_retrieval.py \
--emb evaluation/embeddings_FullTraining_MLH.npy \
--pools evaluation/anchors_pools.json \
--out evaluation/results_FullTraining_MLH.json \
2>&& | tee evaluation/log_eval_FullTraining_MLH.log
{
  "model": "embeddings_FullTraining_MLH.npy",
  "num_anchors_evaluated": 27845,
  "Recall@81": 0.86187052402881,
  "Recall@81@": 0.9448474852792243,
  "MRP": 0.0576724622482681
}
(palmtree-train) ru@pali:~/projects/palmtree$ python evaluation/eval_retrieval.py \
--emb evaluation/embeddings_FullTraining_MLH_OUP.npy \
--pools evaluation/anchors_pools.json \
--out evaluation/results_FullTraining_MLH_OUP.json \
2>&& | tee evaluation/log_eval_FullTraining_MLH_OUP.log
{
  "model": "embeddings_FullTraining_MLH_OUP.npy",
  "num_anchors_evaluated": 27845,
  "Recall@81": 0.863914877141315,
  "Recall@81@": 0.9452788126695316,
  "MRP": 0.0944629864830869
}
(palmtree-train) ru@pali:~/projects/palmtree$ python evaluation/eval_retrieval.py \
--emb evaluation/embeddings_FullTraining_MLH_OUP.npy \
--pools evaluation/anchors_pools.json \
--out evaluation/results_FullTraining_MLH_OUP.json \
2>&& | tee evaluation/log_eval_FullTraining_MLH_OUP.log
{
  "model": "embeddings_FullTraining_MLH_OUP.npy",
  "num_anchors_evaluated": 27845,
  "Recall@81": 0.86669688726881,
  "Recall@81@": 0.946348205307954,
  "MRP": 0.088495277295546
}
(palmtree-train) ru@pali:~/projects/palmtree$

```

Figure 5: Evaluation results

0.6 Challenges

0.6.1 Input Generation Challenges

- Tool-specific differences in IR representations.
- Implicit dependency handling (e.g., EFLAGS).
- Ensuring consistent CFG/DFG extraction.

0.6.2 Training Challenges

- High computational cost.
- Managing multiple model variants.
- Ensuring fair and consistent evaluation.

Addressing these challenges strengthened the reproducibility and robustness of the system.

0.7 Conclusion

This project successfully:

1. Replaced Binary Ninja with angr, creating a fully open-source PalmTree pipeline.
2. Conducted a systematic ablation study on MLM, CWP, and DUP.

The results demonstrate that PalmTree can be made fully free without sacrificing performance. Furthermore, the ablation study provides deeper understanding of how training objectives contribute to instruction embedding quality.

For detailed scripts, training commands, and execution instructions, please refer to the repository and README documentation.